

# Lexical and Syntax Analysis

Programming Languages Intro

Dr. Igor Steinmacher

# Lexical Syntax

- Lowest level of syntax analysis

|                |   |                                                       |
|----------------|---|-------------------------------------------------------|
| Program        | ⇒ | int main ( ) { Declarations Statements }              |
| Declarations   | ⇒ | { Declaration }                                       |
| Declaration    | ⇒ | Type Identifier ;                                     |
| Type           | ⇒ | int   bool   float   char                             |
| Statements     | ⇒ | { Statement }                                         |
| Statement      | ⇒ | ;   Block   Assignment   IfStatement   WhileStatement |
| Block          | ⇒ | { Statements }                                        |
| Assignment     | ⇒ | Identifier = Expression ;                             |
| IfStatement    | ⇒ | if ( Expression ) Statement [ else Statement ]        |
| WhileStatement | ⇒ | while ( Expression ) Statement                        |
| Expression     | ⇒ | Conjunction {    Conjunction }                        |
| Conjunction    | ⇒ | Equality { && Equality }                              |
| Equality       | ⇒ | Relation [ EquOp Relation ]                           |
| EquOp          | ⇒ | ==   !=                                               |
| Relation       | ⇒ | Addition [ RelOp Addition ]                           |
| RelOp          | ⇒ | <   <=   >   >=                                       |
| Addition       | ⇒ | Term { AddOp Term }                                   |
| AddOp          | ⇒ | +   -                                                 |
| Term           | ⇒ | Factor { MulOp Factor }                               |
| MulOp          | ⇒ | *   /   %                                             |
| Factor         | ⇒ | [ UnaryOp ] Primary                                   |
| UnaryOp        | ⇒ | -   !                                                 |
| Primary        | ⇒ | Identifier   IntLit   FloatLit   ( Expression )       |

|            |   |                                   |
|------------|---|-----------------------------------|
| Identifier | ⇒ | Letter { Letter   Digit }         |
| Letter     | ⇒ | a   b   ...   z   A   B   ...   Z |
| Digit      | ⇒ | 0   1   ...   9                   |
| Literal    | ⇒ | Integer   Boolean   Float   Char  |
| Integer    | ⇒ | Digit { Digit }                   |
| Boolean    | ⇒ | true   false                      |
| Float      | ⇒ | Integer.Integer                   |
| Char       | ⇒ | 'ASCIICChar'                      |

# Lexical Syntax

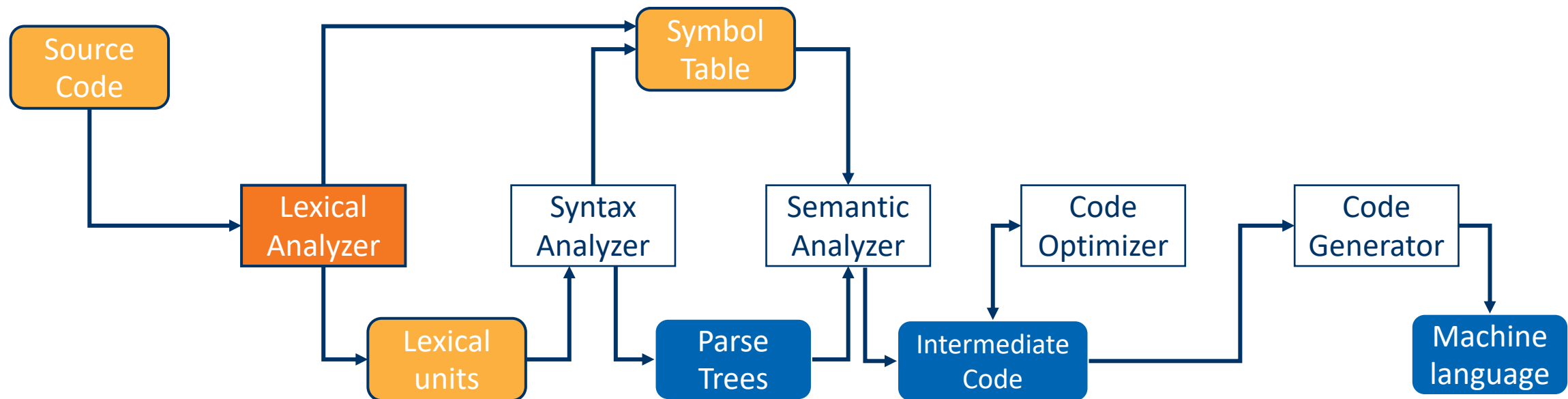
- Lowest level of syntax analysis

This is related to the  
lexical analysis

```
Identifier => Letter { Letter | Digit }
Letter    => a | b | ... | z | A | B | ... | Z
Digit     => 0 | 1 | ... | 9
Literal   => Integer | Boolean | Float | Char
Integer    => Digit { Digit }
Boolean    => true | false
Float      => Integer.Integer
Char       => 'ASCIICChar'
```

# Lexical Syntax

- First step of the compilation



# Lexical Syntax

- Input
  - a stream of characters from the ASCII set, keyed by a developer
- *Output*
  - a stream of *tokens* (lexemes) or basic symbols, classified as, for example:
    - Identifiers    e.g., name, id, x, i, push
    - Literals        e.g., 123, 'x', 3.25, true
    - Keywords       bool char else false float if int main true while
    - Operators      = || && == != < <= > >= + - \* / !
    - Punctuation    ; , { } ( )

# Lexical Syntax

```
index = 2 * count + 17;
```

| Tokens      | Lexemes |
|-------------|---------|
| <ident>     | index   |
| <eq_sign>   | =       |
| <int>       | 2       |
| <mult>      | *       |
| <ident>     | count   |
| <plus>      | +       |
| <int>       | 17      |
| <semicolon> | ;       |

```
<ident> <eq_sign> <int> <mult> <ident> <plus> <int> <semicolon>
```

# Lexical Syntax

- No token contains whitespaces
  - One token: `>=`
  - Two tokens: `> =`
- Whitespaces include
  - Space
  - Tab
  - Return (enter)

# Token identification

- The largest string that constitutes a valid token

while  $x < 2$  do:

# Token identification

- The largest string that constitutes a valid token

while x < 2 do:

Token: <identifier>

# Token identification

- The largest string that constitutes a valid token

`while x < 2 do:`

Token: <identifier>

# Token identification

- The largest string that constitutes a valid token

**while** x < 2 do:

Token: <identifier>

# Token identification

- The largest string that constitutes a valid token

`while` x < 2 do:

Token: <identifier>

# Token identification

- The largest string that constitutes a valid token

**while** x < 2 do:

Token: <whileStatement>

# Token identification

- The largest string that constitutes a valid token

**while** x < 2 do:

Token: <whileStatement>

# Token identification

- The largest string that constitutes a valid token

while x < 2 do:

Token: <whileStatement> <identifier>

# Token identification

- The largest string that constitutes a valid token

while x < 2 do:

Token: <whileStatement> <identifier>

# Token identification

- The largest string that constitutes a valid token

while x < 2 do:

Token: <whileStatement> <identifier> <relOp>

# Token identification

- The largest string that constitutes a valid token

`while`x < 2 do:

Token: <whileStatement>

# Token identification

- The largest string that constitutes a valid token

`whilex` < 2 do:

Token: <identifier>

# Token identification

- The largest string that constitutes a valid token

while x<2 do:

Token: <whileStatement> <identifier>

# Token identification

- The largest string that constitutes a valid token

while x<2 do:

Token: <whileStatement> <identifier> <RelOp>

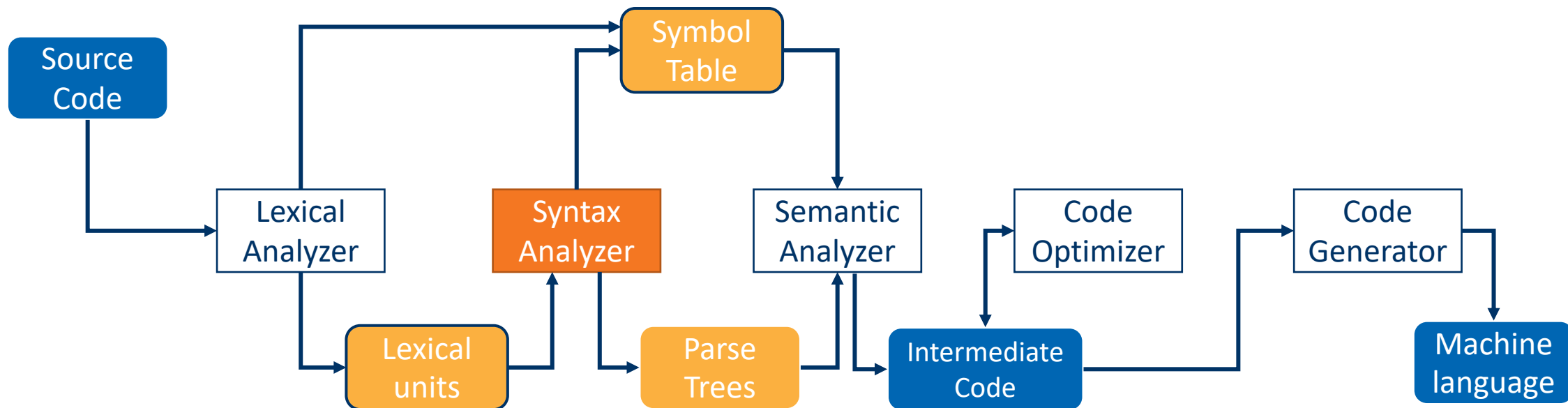
|            |    |        |   |        |  |       |   |   |  |   |  |   |  |     |  |   |
|------------|----|--------|---|--------|--|-------|---|---|--|---|--|---|--|-----|--|---|
| Identifier | => | Letter | { | Letter |  | Digit | } |   |  |   |  |   |  |     |  |   |
| Letter     | => | a      |   | b      |  | ...   |   | z |  | A |  | B |  | ... |  | Z |
| Digit      | => | 0      |   | 1      |  | ...   |   | 9 |  | . |  | . |  | .   |  | . |

# Some more things

- Comments are not defined as part of the grammar
  - Removed before the lexical analyzer (treated as white spaces)
- Reserved words
  - Can be identifiers
  - Defined in a table not kept as part of the grammar

# Abstract Syntax

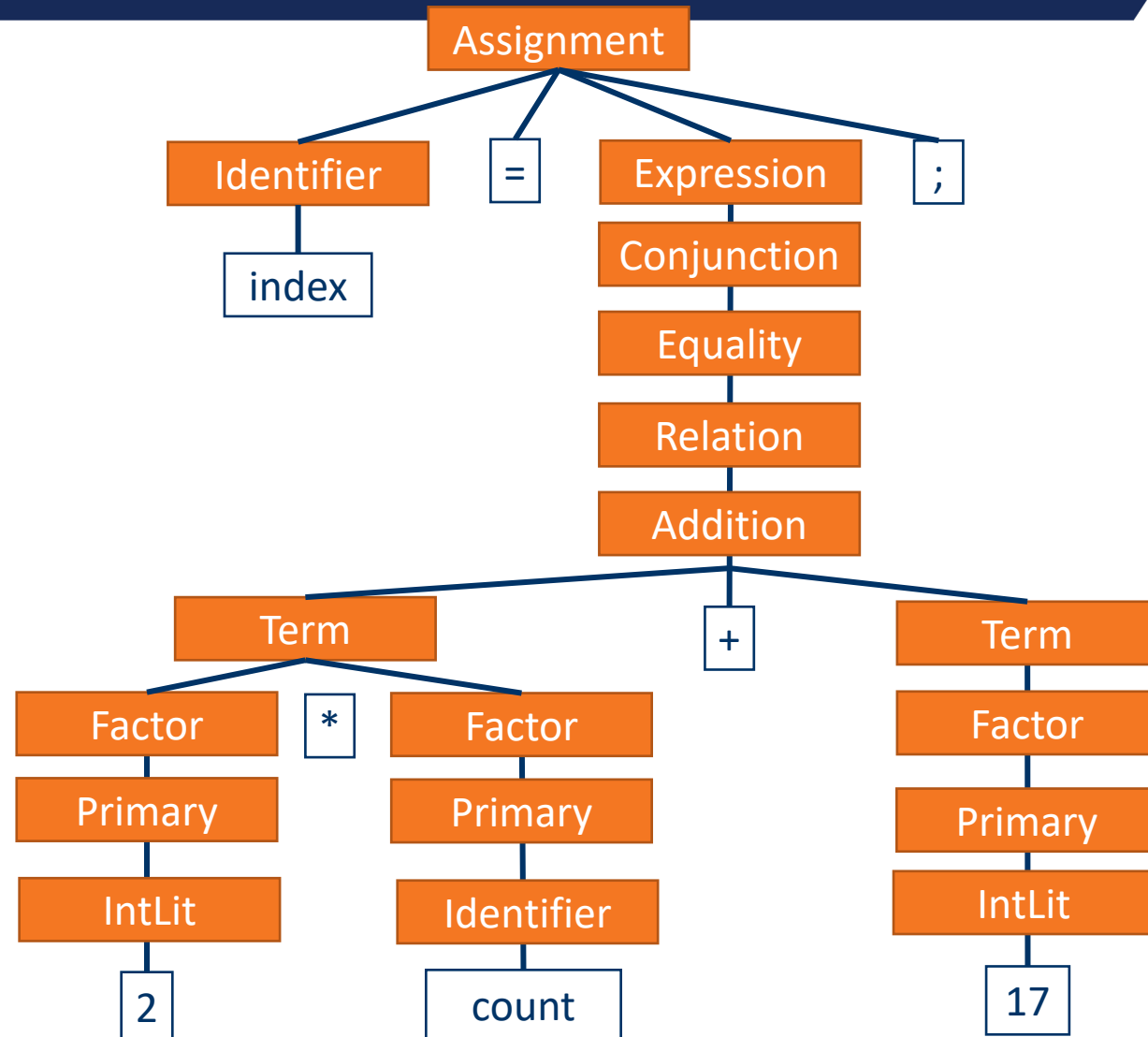
- Based on BNF/EBNF grammar
- Input: tokens
- Output: parse tree → abstract syntax tree



# Syntax Analysis: Parse Tree

- Syntax parse tree is built according to the grammar, considering the lexical units

`index = 2 * count + 17;`



# Abstract Syntax

- Removes “syntactic sugar”
- Keeps essential elements of a language

## Pascal

```
while i < n do begin
    i := i + 1;
end;
```

## C/C++

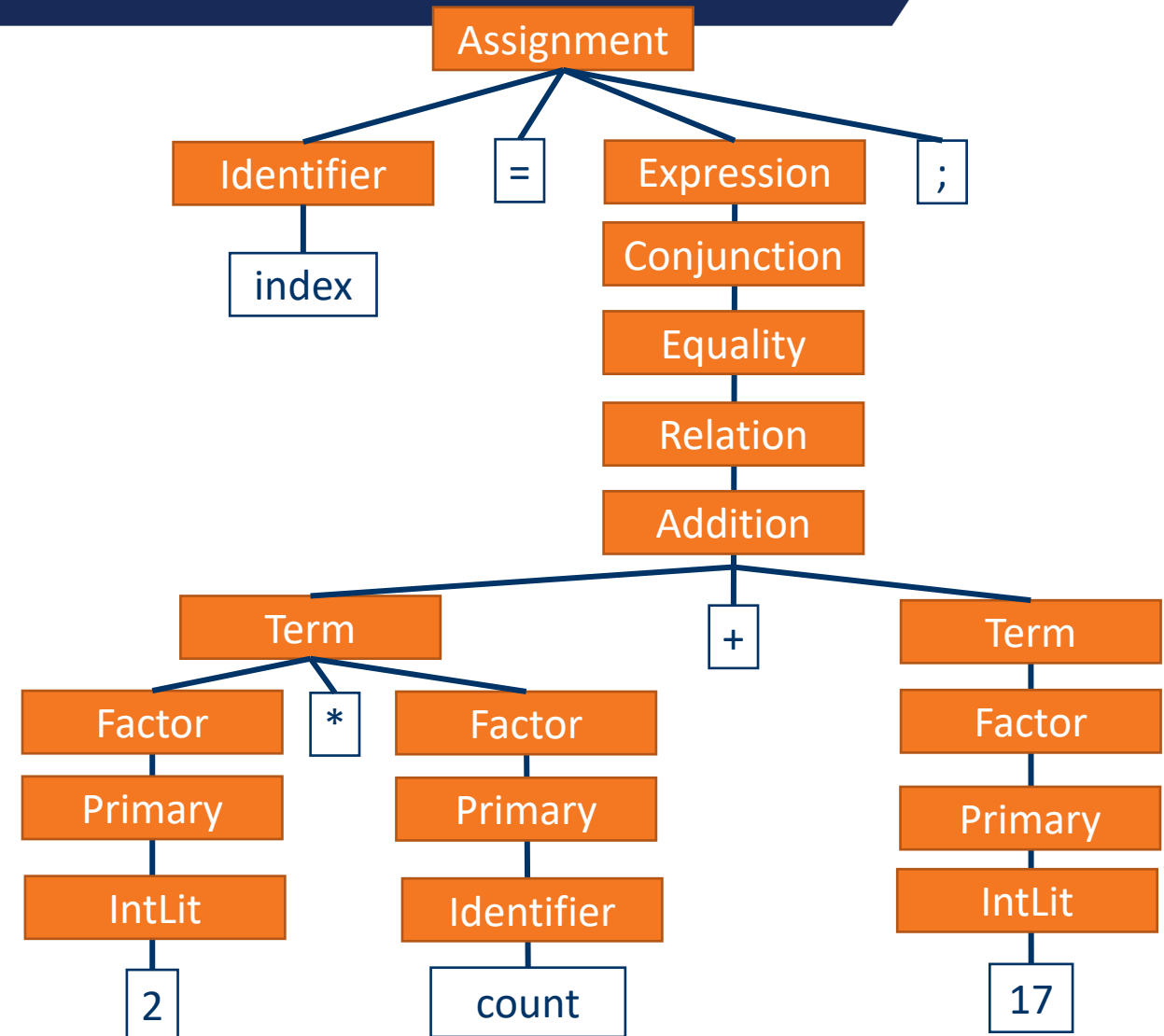
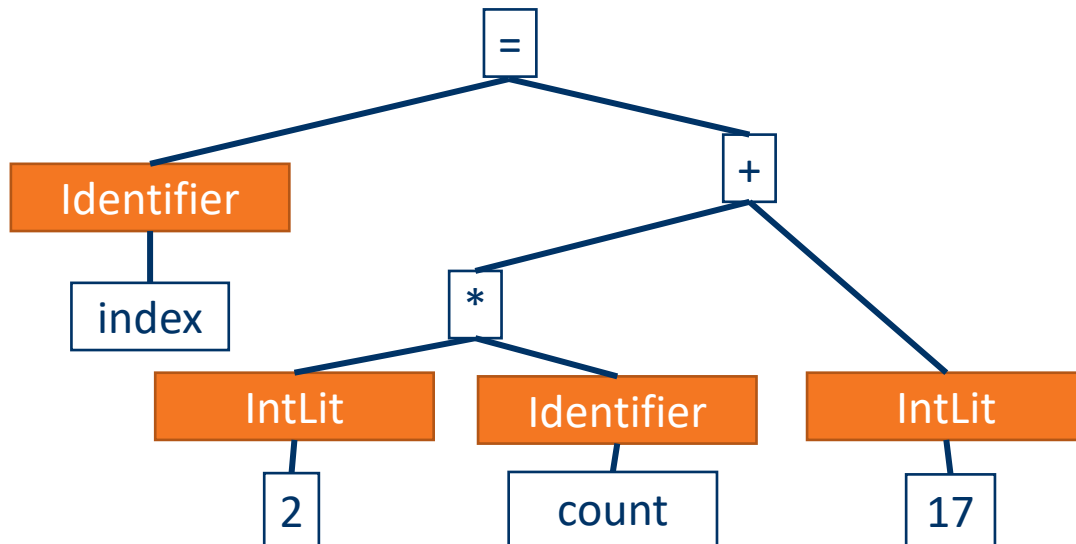
```
while (i < n) {
    i = i + 1;
}
```

The only essential information in each of these:

- 1) it is a *loop*,
- 2) its terminating condition is  $i < n$ ,
- 3) its body increments the current value of  $i$ .

# Syntax Analysis: Abstract Syntax Tree

- Remove punctuation
- Remove trivial derivations
- Replace remaining non-terminals by the operation they represent



**See you in our next  
Lesson!**